SkyChain Intelligence: A Blockchain-Secured Multi-Agent DRL Framework for Low-Altitude Embodied Artificial Intelligence

Haoxiang Luo, Tianqi Jiang, Ruichen Zhang, Yinqiu Liu, Gang Sun, *Senior Member, IEEE*, Hongfang Yu, *Senior Member, IEEE*, Abbas Jamalipour, *Fellow, IEEE*, and Dong In Kim, *Life Fellow, IEEE*

arXiv:2606.24193v1 [cs.NI] 23 Jun 2026

***Abstract*—With the rapid development of the Low-Altitude Economy (LAE) ecosystem, Low-Altitude Embodied Artificial Intelligence (LAEAI) agents have become the core carriers of autonomous aerial services, thereby enabling dynamic Low-altitude Computility Networks (LACNets) for distributed computing resource sharing. However, resource-constrained LAEAI agents in decentralized LACNets face a fundamental trilemma of autonomy, security, and efficiency. Existing solutions primarily focus on either optimizing computational performance or enhancing security in isolation, failing to address the inherent trade-offs among trust, performance, and overhead in untrusted dynamic environments with malicious agents. To tackle this challenge, this paper proposes SkyChain Intelligence, a holistic framework that synergistically integrates agentic AI, consortium blockchain, and Multi-Agent Deep Reinforcement Learning (MADRL). We design a lightweight blockchain-based decentralized trust management system with a dynamic reputation mechanism and develop a hybrid-action-space MADDPG algorithm that embeds on-chain reputation scores into the reward function to jointly optimize offloading decisions, resource allocation, and drone 3D trajectories. Extensive simulations demonstrate that our framework outperforms state-of-the-art baselines in task completion latency and energy consumption, while achieving a 94.1% task completion rate in the baseline scenario and stable convergence within 300 training episodes. This work provides a viable path for building secure, autonomous, and efficient machine-to-machine computing ecosystems in the low-altitude domain.**

***Index Terms*—Agentic AI, Low-Altitude Embodied Artificial Intelligence (LAEAI), Low-Altitude Computility Networks (LACNets), Multi-Agent Deep Reinforcement Learning (MADRL), blockchain.**

I. Introduction

A. Background

THE world is on the cusp of a transformative shift driven by the Low-Altitude Economy (LAE), an emerging economic paradigm centered around various flight activities of civil manned and unmanned aircraft in the airspace typically below 1,000 meters [1]. This nascent domain is poised to unlock immense economic and social value by introducing unprecedented efficiency and capabilities across numerous sectors, including logistics and delivery, precision agriculture, urban air mobility, emergency response, and environmental monitoring [2].

At the heart of this revolution is the advent of Low-Altitude Embodied Artificial Intelligence (LAEAI) [3]. LAEAI refers to autonomous physical agents, such as Unmanned Aerial Vehicles (UAVs) and electric Vertical Take-Off and Landing (eVTOL) aircraft, that can perceive, reason, plan, and act within the physical world. Unlike traditional passive AI, LAEAI does not merely follow preset rules or react to stimuli. It exhibits agentic capabilities, pursuing goals in an autonomous, proactive, and adaptive manner, capable of making independent decisions and executing tasks in dynamic and unstructured environments [4].

Despite the immense potential of LAEAI agents, they are resource-constrained. A single aircraft faces stringent limitations in terms of computational power, onboard energy, sensor range, and communication bandwidth [5]. To execute increasingly complex tasks, such as real-time high-definition video analysis, collaborative environmental mapping, or distributed data processing, individual LAEAI agents must collaborate beyond the confines of their own capabilities [6]. This necessity gives rise to the concept of Low-altitude Computility Networks (LACNets) [7]. We define a LACNet as a dynamic, aircraft-assisted Mobile Edge Computing (MEC) network architecture that provides "computility (computing utility)" on demand. As shown in Fig. 1, in this paradigm, a swarm of aircraft forms a self-organizing aerial computing network where some aircraft can offload their computation-intensive tasks to neighboring aircraft with idle resources [8]. This creates a flexible and resilient computing infrastructure that does not rely on ground-based stations, which is particularly crucial in remote areas or post-disaster scenarios [9].

B. Research Motivation

While the vision of LACNets is promising, its realization is confronted by a fundamental trilemma of autonomy, security, and efficiency.

H. Luo is with the WeBank-NTU Joint Research Institute on Fintech, Nanyang Technological University, Singapore 639798, and also with the College of Computing and Data Science, Nanyang Technological University, Singapore 639798 (e-mail:haoxiang.luo@ntu.edu.sg). T. Jiang is with the School of Science and Engineering, The Chinese University of Hong Kong, Shenzhen 518172, China (e-mail: tikyttqq@gmail.com). R. Zhang and Y. Liu are with the College of Computing and Data Science, Nanyang Technological University, Singapore 639798 (e-mail: ruichen.zhang@ntu.edu.sg; yinqiu001@e.ntu.edu.sg. G. Sun (Corresponding Author) and H. Yu are with the School of Information and Communication Engineering, University of Electronic Science and Technology of China, Chengdu 611731, China (e-mail: {gangsun, yuhf}@uestc.edu.cn). A. Jamalipour is with the School of Electrical and Computer Engineering, University of Sydney, Australia, and with the Graduate School of Information Sciences, Tohoku University, Japan (e-mail: a.jamalipour@ieee.org). D. I. Kim is with the Department of Electrical and Computer Engineering, Sungkyunkwan University, Suwon 16419, South Korea (e-mail: dongin@skku.edu).

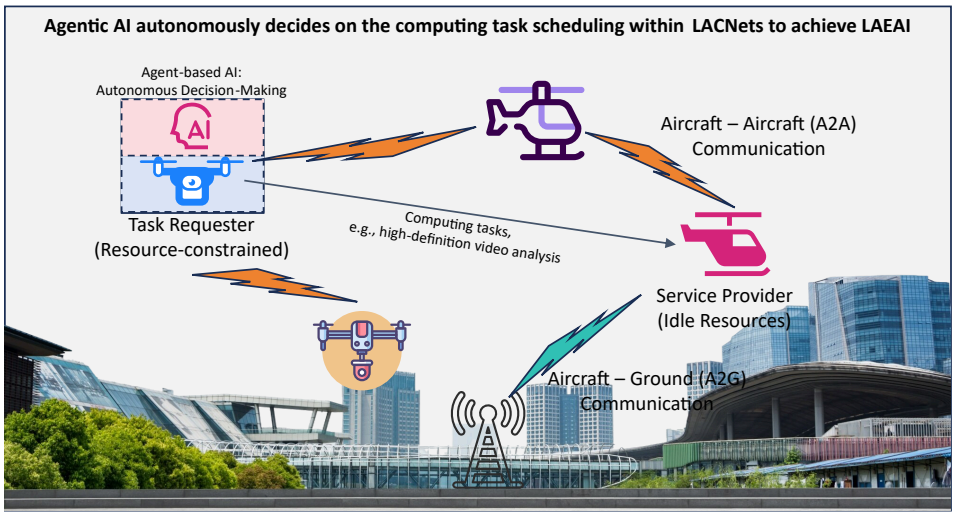


Fig. 1: LAEAI. Each unmanned aircraft will be equipped with agentic AI, which will autonomously determine the computing tasks scheduling in the LACNets to achieve the goals of LAEAI.

First, LAEAI agents must make complex, decentralized decisions in real-time without constant human supervision [10]. This requires them to possess true agentic capabilities, the ability to proactively plan, reason, and act to achieve high-level goals, not just execute pre-programmed instructions [11]. Multi-Agent Deep Reinforcement Learning (MADRL) is a powerful tool for achieving such autonomous decision-making [12], but it requires a well-defined objective function to guide the learning process.

Additionally, in a decentralized network composed of multiple agents, potentially belonging to different operators with competing or selfish behaviors, trust is the cornerstone of collaboration. How can one agent trust another to faithfully execute a computational task, not steal sensitive data, or fairly report its resource usage? Traditional centralized authorities are not only antithetical to the decentralized nature of UAV swarms but also represent a single point of failure [13]. A decentralized trust management mechanism is therefore imperative [14].

Moreover, resource management in a LACNet is an exceedingly complex optimization problem. Jointly optimizing computational resources (CPU cycles), communication resources (spectrum, power), energy (battery life), and agent behavior (trajectory planning, task offloading decisions) constitutes a high-dimensional, non-convex, mixed-integer nonlinear programming problem. It is particularly intractable in a dynamically changing environment [15].

These three challenges are interconnected. An efficient system without security guarantees can have its autonomy exploited by malicious actors. A secure system that is inefficient will fail to meet the real-time demands of LAEAI applications. Therefore, any viable solution must address all three issues concurrently.

C. Our Contributions

To address the trilemma above, this paper proposes a holistic framework named SkyChain Intelligence, which synergistically integrates agentic AI, blockchain, and MADRL. Our framework is built upon a symbiotic relationship where each technology compensates for the inherent limitations of the others in this specific application context. The main contributions of this paper can be summarized as follows:

- **Integrated Framework for Agentic LAEAI:** We propose the SkyChain Intelligence framework, which for the first time synergistically integrates agentic AI, blockchain, and MADRL. This provides a holistic solution that empowers LAEAI agents with the autonomous, goal-driven, and adaptive capabilities needed for secure and efficient computation offloading in LACNets.
- **Blockchain-based Decentralized Trust Management:** We design a trust management system built on a lightweight consortium chain. It features a dynamic reputation scoring mechanism managed by smart contracts and an efficient consensus, enabling reliable cooperation among LAEAI agents without a central authority.
- **Hybrid-Action MADRL for Complex Decision-Making:** We develop a customized MADRL algorithm with a dual-head actor-critic architecture tailored for hybrid action spaces. It enables end-to-end joint optimization of discrete task offloading decisions, continuous resource allocation, and UAV 3D trajectory control in a fully decentralized manner, while embedding blockchain-derived reputation states into the policy learning loop to align agent decision-making with both system performance and trustworthiness.
- **Synergistic Trust-Performance Reward Mechanism:** We introduce a reward function that directly integrates blockchain-derived reputation scores into the MADRL learning process. This creates a powerful feedback loop. It incentivizes agents to learn policies that not only optimize for performance metrics but also actively build and maintain trust within the network.

D. Paper Structure

The remainder of this paper is organized as follows. Section II reviews related work. Section III establishes the detailed

TABLE I: Comparison of Related Works

Related Work	Scenario	Problem	Blockchain	Agentic AI	Hybrid Action	MADRL	3D Trajectory	Resource Alloc.
[5]	3-layer heterogeneous LAENet	Delay and energy optimization under unstable links	✗	✗	✓	✗	✗	✓
[7]	LACNets urban logistics	Trust and resource issues in aerial computility sharing	✓	✗	✗	✗	✗	✓
[8]	Multi-UAV relay and edge networks	Minimize MEC task delay in 3D environments	✗	✗	✓	✓	✓	✓
[16]	UAV relaying in urban environments	Max-min throughput among users	✗	✗	✗	✗	✗	✓
[17]	UAV-assisted FL in remote environments	Minimize FL completion time and mitigate stragglers	✗	✗	✗	✗	✓	✓
[18]	UAV-assisted resource slicing	Joint user association/slicing with interference	✗	✗	✗	✓	✗	✓
[19]	UAV-assisted energy-sensitive tasks	Energy consumption/system complexity in scheduling	✗	✗	✗	✓	✓	✓
[20]	UAV-assisted edge for IoT	Balance energy/delay in multi-device offloading	✗	✗	✗	✗	✗	✓
[21]	UAV-assisted satellite edge computing	Cost minimization under coverage and resource constraints	✗	✗	✗	✗	✗	✓
[22]	Zero-Trust with multi-UAV clusters	Routing security and stability in dynamic UAV networks	✓	✗	✗	✓	✗	✓
SkyChain Intelligence (Ours)	LACNets with LAEAI swarms	Autonomy, security, and efficiency trilemma in MEC offloading	✓	✓	✓	✓	✓	✓

system model and formulates the problem mathematically. Section IV elaborates on the design of the SkyChain Intelligence framework. Section V provides a security analysis of the proposed solution. Section VI presents and analyzes the simulation results in depth. Finally, Section VII concludes the paper and discusses future research directions.

II. Related Works

This section provides a critical review of the existing literature to identify the research gap that our work aims to fill. We will explore four key areas: resource management in aircraft-assisted MEC, DRL for aircraft networks, blockchain for aircraft and IoT security, and Federated Learning (FL) as a comparative approach. Table I presents a comparison of the related work in these fields.

A. Resource Management in Aircraft-Assisted MEC

Aircraft-assisted MEC has been widely studied as an effective paradigm for providing flexible computing services to ground users [8]. Early works primarily focused on using traditional optimization methods to solve resource allocation problems. For instance, some studies employed techniques like convex optimization and block coordinate descent (BCD) to jointly optimize UAV trajectory, power allocation, and task partitioning to minimize latency or energy consumption [16]. Furthermore, game theory has been widely applied to model the competitive or cooperative relationships among aircraft or users [23], seeking optimal resource allocation strategies under Nash Equilibrium (NE) or Stackelberg Equilibrium (SE). These traditional methods perform well in static scenarios where the system model is relatively simple, and the environmental state is precisely known.

The primary limitation of these traditional optimization techniques is their inability to adapt to the highly dynamic and uncertain nature of LACNets [2]. They often rely on complete and accurate global system information, which is impractical to obtain in a decentralized, mobile network of autonomous agents [24]. Moreover, the joint optimization problems are frequently NP-hard, making real-time solutions computationally intractable [5]. This inflexibility and high computational overhead render them unsuitable for enabling the real-time, autonomous decision-making required by LAEAI agents, thereby motivating our adoption of a more adaptive, learning-based DRL approach

B. Deep Reinforcement Learning for LAENets

To overcome the limitations of traditional optimization methods, DRL has emerged as a powerful tool for solving dynamic decision-making problems in LAENets [25]. Researchers have applied various DRL algorithms, including value-based Deep Q-Networks (DQN) and policy-based algorithms like DDPG and Soft Actor-Critic (SAC), to address problems such as aircraft trajectory optimization, computation offloading, and resource allocation [26], [22]. Given the distributed nature of LAENets, MADRL methods, such as MADDPG and MAPPO, have received more attention [27]. Also, the classic MADDPG is incompatible with discrete-continuous hybrid action spaces. These works have successfully demonstrated the great potential of DRL in achieving autonomous decision-making for aircraft and improving system efficiency.

However, a significant and critical flaw in the existing body of DRL-based research is the near-universal assumption of a fully cooperative and trustworthy environment. These models do not account for the possibility of selfish or malicious nodes that may drop tasks, return false results, or act unreliably to

conserve their resources [28]. In an open and economically-driven LAE, where LAEAI agents may belong to different, competing operators, this assumption is unrealistic and dangerous [13]. This critical omission of a trust and security mechanism makes existing DRL solutions vulnerable and incomplete. It directly motivates to integration of a verifiable, decentralized trust layer into the DRL framework.

C. Blockchain for Aircraft and IoT Security

In contrast to DRL, which focuses on efficiency and autonomy, another line of research utilizes blockchain technology to enhance the security of aircraft and Internet of Things (IoT) systems. Numerous studies have explored how to use blockchain to ensure the security of UAV communications, data integrity, reliability of access control, and traceability of transactions [7]. Given the resource constraints of edge devices like aircraft, research often focuses on designing lightweight consensus protocols, such as Practical Byzantine Fault Tolerance (PBFT) and Raft [29] to replace the computationally intensive Proof-of-Work (PoW) [30]. These works have successfully provided a security foundation for decentralized systems by creating immutable and transparent ledgers for transactions and data management [14].

While blockchain provides a robust foundation for security and trust, existing research typically treats it as an isolated component. These studies often focus on static data management and security primitives, failing to integrate the security layer with the dynamic, real-time decision-making processes required for resource management [30]. The inherent latency and energy overhead of blockchain operations are frequently ignored or oversimplified, yet they have a direct impact on the performance of time-sensitive LAEAI applications [31]. This disconnect between security and performance optimization is a major gap. Our framework addresses this by not only using blockchain for trust but also by making the AI agents aware of the blockchain's overhead, allowing them to learn intelligent policies that balance security needs with efficiency constraints.

D. Federated Learning in LAENets

FL is another decentralized AI paradigm applied in LAENets [17]. It allows multiple devices, such as UAVs, to collaboratively train a shared machine learning model without sharing their raw data, thus protecting data privacy and reducing communication overhead [32]. This is particularly useful when data is sensitive or when communication bandwidth is limited, both of which are common constraints in LAENets. The primary goal is to leverage distributed data to build a more accurate and robust global model than any single agent could train on its own [33].

Although both FL and MADRL are decentralized AI technologies, they are designed to solve fundamentally different problems. The core objective of FL is collaborative model training, whereas the core objective of MADRL is collaborative decision control [34]. In our LACNet scenario, the challenge is not to train a shared predictive model, but to enable each LAEAI agent to learn its optimal policy for making sequential decisions about task offloading, resource allocation, and trajectory control [2], [4]. Therefore, this crucial distinction justifies our choice of MADRL as the appropriate framework for empowering LAEAI agents with autonomous decision-making capabilities.

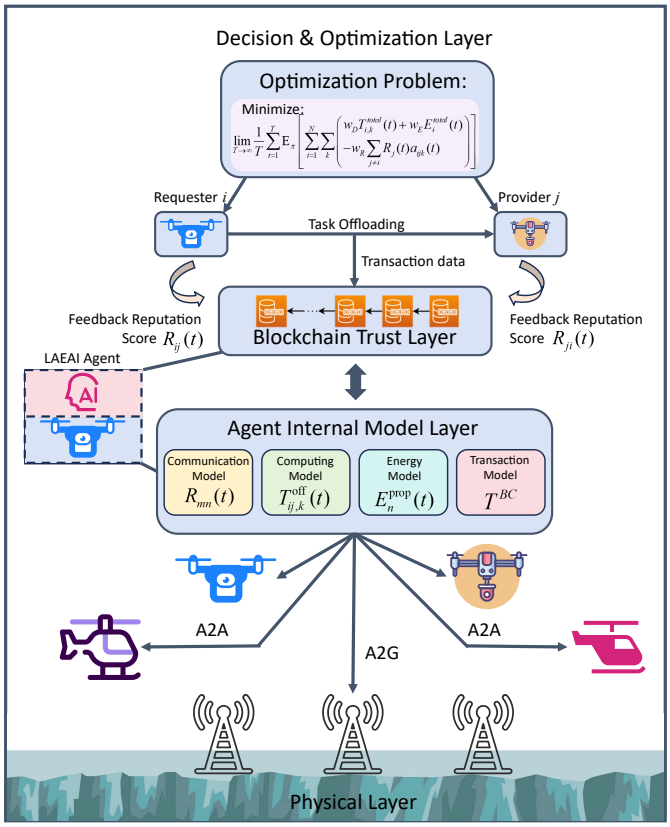


Fig. 2: SkyChain Intelligence framework. It consists of the physical layer for aircraft, the agent layer within the aircraft, the blockchain trust layer, and the final decision and optimization layer.

III. System Model and Problem

This section establishes the rigorous mathematical foundation for our SkyChain Intelligence framework, as shown in Fig. 2, covering the network architecture, communication, computation, energy consumption, and blockchain-based trust models. A summary of key notations is provided in Table II.

A. Network Structure

We consider a Low-altitude Computility Network (LACNet) consisting of a set of N LAEAI agents (aircraft), denoted as $\mathcal{N} = \{1, ..., N\}$. These UAVs operate in a three-dimensional Cartesian space over a finite time horizon T, which is discretized into time slots $t \in \{1, ..., T\}$. The position of aircraft n at time slot t is denoted by $\mathbf{q}_n(t) = [x_n(t), y_n(t), H_n]$, where H_n is its quasi-static flight altitude. The network also includes a set of M ground task generators, e.g., IoT devices or ground UAV stations, denoted as $\mathcal{M} = \{1, ..., M\}$, with fixed positions $\mathbf{w}_m$. In this system, aircraft can act as both requesters of computation tasks and providers of computation services, forming a peer-to-peer computation market.

TABLE II: Summary of Notations

Symbol	Definition/Physical Meaning
$a_{ijk}(t)$	Binary offloading decision variable
B	Communication channel bandwidth
C_k	Computation density of task k
D_k	Data size of task k
E^{BC}, T^{BC}	Energy and latency for a blockchain transaction
E^{comp}, E^{trans}	Energy for computation and communication
E^{prop}, P^{prop}	Energy and power for aircraft propulsion
f_i^{loc}, f_j^{edge}	CPU frequency for local and edge computation
H_n	Flight altitude of aircraft n
$\mathcal{M}, M$	Set and number of ground task generators
$\mathcal{N}, N$	Set and number of LAEAI agents (aircraft)
N_0	Noise power spectral density
$o_n(t), a_n(t), r_n(t)$	Observation, action, and reward for agent n
$P_{LoS}(\cdot)$	Probability of a LoS link
P_m, P_n^{trans}	Transmit power of device m and aircraft n
$PL_{nm}(t)$	Path loss between aircraft n and device m
Q_{ϕ_n}	Actor and Critic networks for agent n
$R_{mn}(t)$	Data transmission rate from m to n
$\mathcal{R}_n(t)$	Reputation score of aircraft n at time t
t, T	Time slot index and total number of time slots
$T_{i,k}^{loc}, T_{ij,k}^{off}$	Latency for local and offloaded computation
$T_k^{\max}$	Max latency of task k
w_D, w_E, w_R	Weighting factors for delay, energy, and reputation
$\mathbf{w}_m$	Fixed 3D position of ground device m
$\mathbf{q}_n(t)$	3D position of aircraft n at time t
$\theta_{nm}(t)$	Elevation angle between aircraft n and device m
μ_{θ_n}	Actor network for agent n

B. Air-to-Ground (A2G) Communication Model

In complex low-altitude environments such as cities, a deterministic Line-of-Sight (LoS) channel model is unrealistic [8]. Therefore, we adopt a more realistic probabilistic LoS channel model. The probability of an LoS link between aircraft n at position $\mathbf{q}_n(t)$, and ground device m at position $\mathbf{w}_m$ depends on the elevation angle $\theta_{nm}(t)$ between them, namely,

$$\theta_{nm}(t) = \arctan\left(\frac{H_n}{\|\mathbf{q}_n^{xy}(t) - \mathbf{w}_m\|}\right), \tag{1}$$

where $\mathbf{q}_n^{xy}(t)$ is the projection of aircraft n onto the 2D plane.

According to the model in [35], the LoS probability can be expressed as a sigmoid function,

$$P_{LoS}(\theta_{nm}(t)) = \frac{1}{1 + a\exp(-b(\theta_{nm}(t) - a))}, \tag{2}$$

where a and b are environment-specific parameters, e.g., urban, suburban, that reflect the statistical properties of building density and height.

Additionally, the total path loss $PL_{nm}(t)$ is a weighted average of the LoS and Non-Line-of-Sight (NLoS) path losses [8], that is,

$$\begin{aligned} PL_{nm}(t) =& P_{LoS}(\theta_{nm}(t)) \cdot PL_{nm}^{\text{LoS}}(t) \\ &+ (1 - P_{LoS}(\theta_{nm}(t))) \cdot PL_{nm}^{\text{NLoS}}(t) \end{aligned} \tag{3}$$

where the mean path losses (in dB) for LoS and NLoS links are given by

$$\begin{aligned} \bar{PL}_{nm}^{\text{LoS/NLoS}}(t) =& 20\log_{10}(d_{nm}(t)) + 20\log_{10}(f_c) \\ &+ 20\log_{10}\left(\frac{4\pi}{c}\right) + \eta_{\text{LoS/NLoS}}, \end{aligned} \tag{4}$$

$$d_{nm}(t) = \|\mathbf{q}_n(t) - \mathbf{w}_m\|, \tag{5}$$

where $d_{nm}(t)$ is the distance between the aircraft and the ground device, f_c is the carrier frequency, c is the speed of light, and η_{LoS} and η_{NLoS} are additional attenuation factors for LoS and NLoS links, respectively.

Building upon this mean path loss formulation, we model long-term shadowing loss due to random building blockages as a zero-mean log-normal random variable:

$$PL_{nm}^{\text{LoS/NLoS}}(t) = \bar{PL}_{nm}^{\text{LoS/NLoS}}(t) + \xi_{\text{LoS/NLoS}}, \tag{6}$$

where $\xi_{\text{LoS/NLoS}} \sim \mathcal{N}(0, \sigma^2_{\text{LoS/NLoS}})$ denotes shadowing fluctuation in the dB domain [36]. The terms η_{LoS} and η_{NLoS} in Eq. (4) represent the mean additional attenuation, including average shadowing and clutter loss, for each link state. The probabilistic LoS/NLoS switching in Eqs. (2)–(3) further captures long-term blocking state transitions driven by UAV mobility.

Moreover, according to the Shannon-Hartley theorem, the achievable data rate for offloading task k from device m to aircraft n is

$$R_{mn}(t) = B\log_2\left(1 + \frac{P_m|h_{mn}(t)|^2}{N_0}\right), \tag{7}$$

where B is the channel bandwidth, P_m is the transmission power of device m, N_0 is the noise power spectral density.

We further decompose the instantaneous channel gain to characterize short-term small-scale fading:

$$|h_{mn}(t)|^2 = 10^{-PL_{nm}(t)/10} \cdot \alpha^2(t) \tag{8}$$

where $|h_{mn}(t)|^2$ is the channel gain, $\alpha(t)$ is the small-scale fading envelope. We define the Rician K-factor as the power ratio of the LoS component to scattered multipath components $K = \frac{P_{\text{LoS}}}{P_{\text{scatter}}}$. For LoS paths, $\alpha(t)$ follows a Rician distribution with an elevation-dependent K; for NLoS paths, $\alpha(t)$ reduces to a Rayleigh distribution ($K = 0$) due to rich multipath scattering [35].

Moving beyond A2G links, Air-to-Air (A2A) channels between UAVs exhibit distinct propagation properties due to the elevated position of both transceivers. They are typically modeled as Rician fading channels with a much higher LoS probability [37]. Also, we denote the Rician K-factors of A2G and A2A LoS links as K_{A2G} and K_{A2A}, respectively. K_{A2G} varies with elevation angle due to ground clutter, while K_{A2A} remains stable as both transceivers operate above rooftop level with fewer obstacles [38].

For high-mobility scenarios where UAVs move independently, the maximum Doppler shift induced by radial relative motion is formulated as $f_d^{\max} = \frac{v_r f_c}{c}$, where v_r m/s denotes the maximum radial relative velocity. The corresponding channel coherence time follows the standard approximation $T_c \approx \frac{9}{16\pi f_d^{\max}}$, which is far longer than the slot duration Δt (i.e., quasi-static fading model) for the considered system carrier frequency [39]. As a result, Doppler variations are fully captured by updating channel states slot-by-slot with real-time UAV positions.

C. Computation Task and Offloading Model

Each computation task k initiated or received by aircraft n is defined by a tuple $\mathcal{T}_k = \{D_k, C_k, T_k^{\max}\}$, representing the task's data size (in bits), computation density (CPU cycles/bit), and maximum tolerable latency (in seconds), respectively.

We define a discrete offloading decision variable $a_{ijk}(t) \in \{0,1\}$, where $a_{ijk}(t) = 1$ indicates that aircraft i offloads task k to aircraft j for computation at time slot t, with $\sum_{j\in\mathcal{N}} a_{ijk}(t) = 1$. The case $j = i$ represents local computation. Then, we set the local computation latency of aircraft n for task k to be

$$T_{i,k}^{\text{loc}}(t) = \frac{D_k C_k}{f_i^{\text{loc}}}, \tag{9}$$

where f_i^{loc} is the CPU frequency of aircraft i used for local computation. The total computation latency for offloading to aircraft j includes transmission latency and computation latency at aircraft j, namely,

$$T_{ij,k}^{\text{off}}(t) = T_{ij,k}^{\text{trans}}(t) + T_{j,k}^{\text{comp}}(t) = \frac{D_k}{R_{ij}(t)} + \frac{D_k C_k}{f_j^{\text{edge}}}, \tag{10}$$

where $R_{ij}(t)$ is the transmission rate from aircraft i to j, and f_j^{edge} is the CPU frequency allocated by aircraft j to this task.

D. Energy Consumption Model

The total energy consumption of the system consists of computation, communication, flight, and blockchain interaction.

First, for the computation energy, the processing task k is proportional to the square of the CPU frequency,

$$E_{n,k}^{\text{comp}} = \kappa (f_n)^2 D_k C_k \tag{11}$$

where κ is the effective capacitance coefficient related to the processor chip architecture.

Second, the communication energy for task k is

$$E_{ij,k}^{\text{trans}} = P_i^{\text{trans}} \cdot T_{ij,k}^{\text{trans}}(t) \tag{12}$$

where P_i^{trans} is the transmission power of aircraft i.

Third, for a rotary-wing aircraft, the aircraft propulsion energy is a complex function of its flight speed $v_n(t)$ [40], that is

$$\begin{aligned} P_n^{\text{prop}}(v_n(t)) =& P_0 \left(1 + \frac{3v_n(t)^2}{U_{\text{tip}}^2}\right) \\ &+ P_i \left(\sqrt{1 + \frac{v_n(t)^4}{4v_0^4}} - \frac{v_n(t)^2}{2v_0^2}\right)^{1/2} + \\ &\frac{1}{2} d_0 \rho s A v_n(t)^3, \end{aligned} \tag{13}$$

where P_0 and P_i are the blade profile power and induced power in hover, respectively, U_{tip} is the blade tip speed, v_0 is the mean rotor induced velocity in hover, and others are aerodynamic constants. The propulsion energy within time slot Δt is

$$E_n^{\text{prop}}(t) = P_n^{\text{prop}}(v_n(t)) \cdot \Delta t. \tag{14}$$

Finally, we model blockchain transaction energy as the sum of fixed local validation energy and dynamic communication energy, to capture the impacts of wireless channel fading and UAV mobility. Local validation energy $E_{\text{comp}}^{\text{BC}}$ (signature verification, smart contract execution) remains a constant.

We define $R_{\min}$ as the minimum A2A rate for delivering one consensus message per slot. An outage occurs when $R_n(t) < R_{\min}$ and triggers retransmissions. For A2A Rician channels, the outage probability has a closed-form expression [41]:

$$P_{\text{out},n}(t) = 1 - Q_1\left(\sqrt{2K_{\text{A2A}}}, \sqrt{2(K_{\text{A2A}} + 1) \cdot \frac{\gamma_{\text{th}}}{\gamma_n(t)}}\right), \tag{15}$$

where $Q_1(\cdot,\cdot)$ is the first-order Marcum Q-function, $\gamma_{\text{th}} = 2^{R_{\min}/B} - 1$ is the Signal-to-Noise Ratio (SNR) threshold, and $\gamma_n(t)$ is the received SNR at the time slot t with the quasi-static fading model.

Incorporating expected retransmission overhead for wireless consensus, the per-transaction communication energy for node n in a shard of size N_s at slot t is:

$$E_{\text{trans},n}^{\text{BC}}(t) = \frac{N_s \cdot P_n^{\text{trans}} \cdot \frac{S_{\text{msg}}}{R_n(t)}}{1 - P_{\text{out},n}(t)}, \tag{16}$$

where S_{msg} is the consensus message size, and $R_n(t)$ is the achievable A2A rate from Section III-B. The denominator denotes the expected number of transmission attempts for successful delivery. Thus, total blockchain energy can be expressed as $E_n^{\text{BC}}(t) = E_{\text{comp}}^{\text{BC}} + E_{\text{trans},n}^{\text{BC}}(t)$.

E. Blockchain-based Trust and Transaction Model

We adopt a consortium chain architecture, where all participating LAEAI agents are permissioned nodes. This provides better privacy, controllability, and performance than a public chain. To minimize latency and energy overhead, we choose a lightweight consensus protocol, namely the sharded Symbiotic PBFT (SS-PBFT) [13]. In this consensus, a symbiotic and reciprocal transmission relationship was established between the nodes, and the consensus efficiency was significantly improved and energy consumption was reduced through the form of sharding.

Additionally, a key smart contract is used to manage the reputation score $\mathcal{R}_n(t) \in [0,1]$ of each aircraft n. After aircraft j completes a computation task for aircraft i, the task requester i submits feedback (e.g., 1 for success, 0 for failure) to the smart contract. We design a dual-track reputation mechanism, which is elaborated as follows: *1) Short-term reputation* $\mathcal{R}_j^{\text{s}}(t)$ adopts a large update factor $\beta_s \in (0,1]$ to sensitively capture recent behavioral changes, enabling rapid identification of degraded or malicious nodes; *2) Long-term reputation* $\mathcal{R}_j^{\text{l}}(t)$ adopts a small update factor $\beta_l \in (0,1]$ ($\beta_l < \beta_s$) to accumulate historical interaction records, anchoring reputation to long-term performance to resist intermittent misbehavior and whitewashing.

The update rules are:

$$\mathcal{R}_j^{\text{s}}(t) = \begin{cases} (1-\beta_s)\mathcal{R}_j^{\text{s}}(t-1) + \beta_s \cdot \text{Feedback}, & \text{If Normal,} \\ \mathcal{R}_j^{\text{s}}(t-1) - \beta_s \cdot \mathcal{R}_{\text{penalty}}, & \text{If Malicious,} \\ \mathcal{R}_{\text{init}}, & \text{New Node,} \end{cases} \tag{17}$$

$$\mathcal{R}_j^{\text{l}}(t) = (1-\beta_l)\mathcal{R}_j^{\text{l}}(t-1) + \beta_l \cdot \text{Feedback}, \tag{18}$$

where Feedback $\in \{0,1\}$. $\mathcal{R}_{penalty} \in (0,1]$ represents the penalty coefficient for malicious behavior. $\mathcal{R}_{init}$ is the initial reputation of the new node, which prevents the new node from immediately obtaining high trust privileges. The final comprehensive reputation score is the weighted fusion of the two tracks:

$$\mathcal{R}_j(t) = \lambda \mathcal{R}_j^{\text{short}}(t) + (1-\lambda)\mathcal{R}_j^{\text{long}}(t) \tag{19}$$

where $\lambda \in (0,1)$ balances sensitivity to immediate behavior and reliance on long-term credit.

Meanwhile, a computation offloading task is encapsulated as a transaction and recorded on the blockchain. The transaction content includes {`RequesterID`, `ProviderID`, `TaskID`, `ResourcePrice`, `ComputationResultHash`}. This provides an immutable audit trail for all interactions.

Then, the total latency for a blockchain transaction is

$$T^{\text{BC}} = T^{\text{prop}} + T^{\text{consensus}}, \tag{20}$$

where T^{prop} is the network propagation delay and $T^{\text{consensus}}$ is the time required to reach consensus. For the SS-PBFT, the consensus time is typically in the order of seconds due to the small scope of consensus, which is much lower than that of the original PBFT. To adapt to high UAV mobility and frequent topology changes, we add two mobility-aware optimizations to SS-PBFT:

1) Topology-aware dynamic sharding: Shards are re-clustered per time slot via greedy clustering [14], where only node pairs with A2A LoS probability $P_{\text{LoS}}(t) \geq P_{\text{th}}^{\text{shard}}$ are grouped into the same shard to stabilize intra-shard links. The highest-reputation, lowest-mobility node is elected as shard primary to reduce coordinator failures.

2) Checkpoint-based fast synchronization: Nodes generate compact state checkpoints every M blocks. Reconnected nodes only sync incremental blocks after the latest checkpoint instead of the full ledger. For temporary partitions, shards run independent consensus, and ledgers are merged by reputation-weighted voting upon reconnection to ensure eventual consistency.

F. Adversarial Model

This part formally defines the adversary capabilities and types, and attack objectives of the system. It provides a unified theoretical framework for subsequent security analysis and algorithm design.

1) Adversary Capabilities: We define the adversary as attackers who possess the following capabilities:

- **Task Execution Tampering Capability:** The adversary can control the malicious node j. After accepting the offloaded task k, it discards the task with probability $p_{drop} \in [0,1]$, or returns a tampered, incorrect calculation result and forged result hash with probability $p_{tamper} \in [0,1]$ [42].
- **Sybil Attack Capability:** The adversary can try to forge multiple fake node identities to access the network [43]. However, it is limited by the admission mechanism of the consortium blockchain, and can only initiate identity forgery through authorized malicious nodes, and cannot access anonymously. Meanwhile, it can collude with multiple controlled malicious nodes to manipulate the reputation feedback of the system.
- **Free-Riding Capability:** The adversary can control rational selfish nodes [44]. After accepting the task, it executes the task with the minimum computing power or refuses to execute the task to save its own computing, communication, and flight energy consumption. While it maintains the online state of the node to accept more tasks.
- **Denial of Service (DoS) Attack Capability:** The adversary can control malicious nodes to flood a large number of fake/invalid computing task requests to high-reputation honest service nodes [45]. Thus, it can exhaust the computing and communication resources of the target node, resulting in congestion of its task queue and a sharp increase in processing delay.

2) Adversary Types: Based on the behavior, and attack mode of the adversary, the malicious nodes controlled by the adversary are divided into two categories:

- **Byzantine Malicious Adversary:** Corresponding to the malicious aircraft, its core goal is to actively destroy the security and availability of the system, and launch all the above four types of attacks indiscriminately.
- **Rational Selfish Adversary:** Corresponding to the free-riding nodes, its core goal is to maximize its own resource income and minimize its own energy consumption. It only initiates selfish free-riding attacks, and will not actively destroy the system.

G. Problem Formulation

Our objective is to find an optimal policy π^* that minimizes the long-term, weighted total cost of the system, which is a combination of task completion latency, total energy consumption, and a negative trust-weighted reward. This reflects a core trade-off in the system, which we term the "Trust-Performance-Overhead" trilemma.

1) Performance vs. Overhead: To pursue extremely low latency, an aircraft might prefer to offload tasks to its nearest neighbor. However, recording this transaction on the blockchain for security introduces additional latency and energy overhead, thereby reducing overall performance.

2) Trust vs. Performance: The aircraft with the highest reputation may not be the geographically closest or have the most available computational resources. Choosing it might increase latency and energy consumption, but ensures the secure and reliable completion of the task. Conversely, choosing a nearby but low-reputation node is risky but potentially faster.

3) Trust vs. Overhead: To maintain a high-precision trust evaluation, more frequent on-chain interactions are needed to update reputation scores, but this directly increases the total overhead of the blockchain system.

Additionally, the optimal decision is not static but highly dependent on the current system state, e.g., task urgency, node reputation, and network congestion. Therefore, our optimization goal is to learn a policy that can dynamically balance these competing factors.

Objective Function:

$$\min_{\pi} \lim_{T\to\infty} \frac{1}{T} \sum_{t=1}^{T} \mathbb{E}_{\pi} \Bigg[\sum_{i=1}^{N} \sum_{k} \Bigg(w_D T_{i,k}^{\text{total}}(t) + w_E E_i^{\text{total}}(t) - w_R \sum_{j\neq i} \mathcal{R}_j(t) a_{ijk}(t) \Bigg) \Bigg] \tag{21}$$

where $T_{i,k}^{\text{total}}(t)$ is the end-to-end total latency for task k originating from agent i, $E_i^{\text{total}}(t)$ is the total energy consumption of aircraft i at time slot t (including propulsion, computation, communication, and blockchain), $\mathcal{R}_j(t)$ is the reputation of the service provider j, and $a_{ijk}(t)$ is the offloading decision indicator. w_D, w_E, w_R are weighting factors to balance the importance of latency, energy, and trust.

Constraints:

$$\begin{aligned}
&(C1): \|\mathbf{q}_n(t+1) - \mathbf{q}_n(t)\| \leq v_{\max}\Delta t, \quad \forall n \in \mathcal{N}, \\
&(C2): \|\mathbf{q}_i(t) - \mathbf{q}_j(t)\| \geq d_{\min}, \quad \forall i \neq j \in \mathcal{N}, \\
&(C3): \sum_{j\in\mathcal{N}} a_{ijk}(t) = 1, \quad \forall i,k, \\
&(C4): f_n(t) \leq f_n^{\max}, \quad P_n^{\text{trans}}(t) \leq P_n^{\max}, \quad \forall n \in \mathcal{N}, \\
&(C5): \Pr\left(T_{i,k}^{\text{total}}(t) \leq T_k^{\max}\right) \geq \varepsilon_k, \quad \forall i,k.
\end{aligned} \tag{22}$$

1) Collision Avoidance / Safety Distance Constraint $(C2)$*:* The left side of the inequality calculates the distance traveled by aircraft n within a single time slot Δt. The right side represents the maximum possible distance the aircraft can travel within that time slot at its maximum speed $v_{\max}$. Therefore, this constraint ensures that the aircraft's trajectory is physically feasible, adhering to its inherent speed limitations.

2) Aircraft Kinematics Constraint $(C1)$*:* The left side of the inequality represents the Euclidean distance between any two distinct aircraft, i and j. $d_{\min}$ is the predefined minimum safety distance. This constraint is fundamental to the safe operation of the LACNets to prevent mid-air physical collisions.

3) Task Assignment Uniqueness Constraint $(C3)$*:* The summation over all possible computing nodes j, including the agent i itself, being equal to 1 ensures that each computation task is assigned to one and only one location for processing. This prevents task duplication and ensures that no task is dropped during the assignment process.

4) Hardware / Resource Limitation Constraint $(C4)$*:* This constraint reflects the physical hardware limitations of each aircraft. The CPU frequency used by aircraft n, $f_n(t)$, cannot exceed its processor's maximum clock speed, $f_n^{\max}$. Likewise, its transmission power, $P_n^{\text{trans}}(t)$, is capped by the maximum power supported by its communication module, $P_n^{\max}$. This ensures that the framework's decisions are practical and operate within the feasible capabilities of the agents' hardware.

5) Quality of Service (QoS) Constraint $(C5)$*:* $T_{i,k}^{\text{total}}(t)$ represents the total end-to-end latency for completing task k. $T_k^{\max}$ is the predefined maximum tolerable deadline for that specific task. We adopt a chance-constrained formulation, where $\varepsilon_k \in (0,1]$ denotes the required completion reliability for task k. This formulation guarantees long-term service quality while tolerating instantaneous violations caused by fading.

IV. SkyChain Intelligence Solution

To solve the complex optimization problem formulated in the previous section, we design the SkyChain Intelligence framework. This framework empowers LAEAI agents with agentic AI capabilities to make autonomous decisions.

A. The Role of Agentic AI

Agentic AI refers to systems that can autonomously and proactively pursue goals with limited human supervision by perceiving their environment, reasoning, planning, and executing actions [4], [46]. In our framework, each LAEAI agent is an instance of agentic AI. Its agentic nature is not a pre-programmed set of rules but an emergent property derived from the learning process [47]. The MADRL algorithm is the engine that drives this agentic behavior, as shown in Fig. 3.

1) Perception: The agent perceives its local environment through its observation space $o_n(t)$, which includes its physical state, network conditions, task queues, and trust information from the blockchain.

2) Reasoning and Planning: The actor-critic neural networks process these observations to reason about the complex trade-offs. The agent implicitly plans a sequence of actions, such as trajectory adjustments and offloading decisions, to maximize its long-term cumulative reward.

3) Execution: The agent executes its plan by outputting a hybrid action $a_n(t)$ that directly controls its physical movement and resource management decisions.

4) Learning and Adaptation: By receiving reward feedback and observing the consequences of its actions, the agent continuously adapts its internal policy, improving its ability to achieve its goals in a dynamic and unpredictable environment.

This approach enables LAEAI agents to move beyond simple automation and exhibit intelligent, goal-directed behavior in the complex low-altitude domain.

B. Agent Decision Process as a Dec-POMDP

Since each agent has only partial information about the global state and its decisions impact the entire system, we model the problem as a Dec-POMDP defined by the tuple $\langle \mathcal{N}, S, \{A_n\}_{n\in\mathcal{N}}, T, R, \{O_n\}_{n\in\mathcal{N}}, \gamma \rangle$ [48]:

1) State Space (S)*:* The global state $s(t) \in S$ includes all UAVs' positions, velocities, task queues, remaining energy, and the blockchain's current reputation scores.

2) Observation Space (O_n)*:* At time slot t, each agent n's local observation $o_n(t) \in O_n$ is a subset of the global state, including its own status, information about its neighbors, current task properties, and a local copy of the reputation table.

3) Action Space (A_n)*:* Each agent's action space is hybrid, comprising i) Discrete Action a_n^d: The task offloading decision, selecting a computation node j from its neighbors or itself; ii) Continuous Action a_n^c: Trajectory control (velocity vector $[v_x, v_y]$), transmit power P_n^{trans}, and CPU frequency f_n^{alloc}.

4) Transition Probability (T)*:* $T(s'|s,\mathbf{a})$ denotes the probability of transitioning from state s to s' after the joint action $\mathbf{a} = (a_1, \ldots, a_N)$.

5) Reward Function (R_n)*:* Each agent n receives a local reward, as shown below:

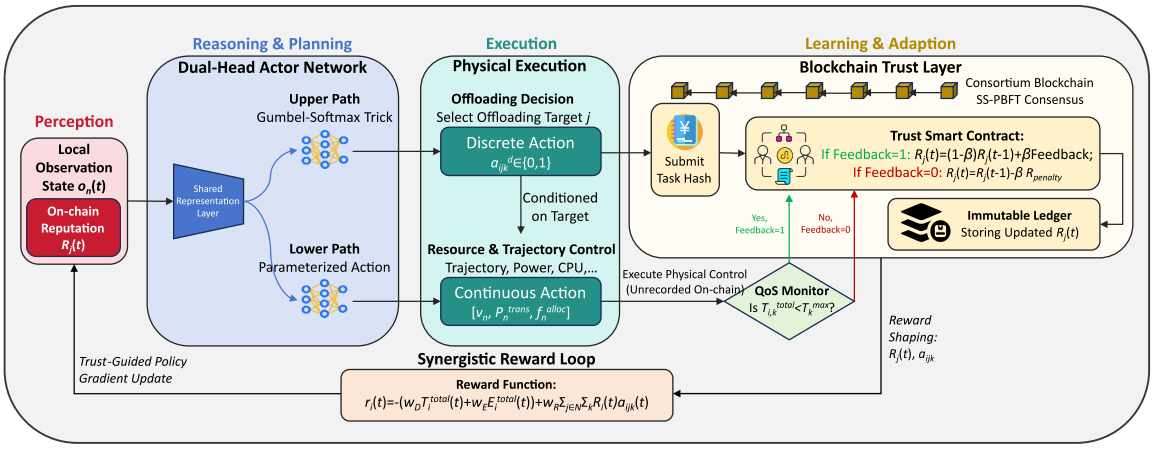


Fig. 3: Integrating Agentic Paradigms and Blockchain Trust. It reveals the four LAEAI stages of "perception-reasoning-execution- adaptation" in the SkyChain framework, deeply coupling the continuous optimization of underlying physical resources with the discrete blockchain reputation reward mechanism.

$$r_i(t) = -\underbrace{(w_D T_i^{total}(t) + w_E E_i^{total}(t))}_{\text{Basic Performance Cost Term}} + \underbrace{w_R \sum_{j\in\mathcal{N}} \sum_{k} \mathcal{R}_j(t) a_{ijk}(t)}_{\text{Trust Incentive Term}}, \tag{23}$$

6) Discount Factor (γ): A discount factor $\gamma \in [0, 1)$ that balances immediate and future rewards.

C. Hybrid Action Space MADDPG Algorithm

We adapt the MADDPG algorithm, which follows the Centralized Training, Decentralized Execution (CTDE) paradigm, to handle our hybrid action space using a parameterized action space approach.

1) Actor Network Architecture: Each agent's actor network μ_{θ_n} has two output heads: a **discrete action head** that uses the Gumbel-Softmax trick to select an offloading target, and a **continuous parameter head** that outputs the continuous control parameters (trajectory, power, CPU) conditioned on the selected discrete action. The former's decision will be recorded on the blockchain, while the latter will only write the judgment result of $T_{i,k}^{\text{total}}(t) \leq T_k^{\max}$ onto the chain. It can avoid the overhead of recording a large number of decisions on the chain. To mitigate the off-chain continuous parameter falsification risk, we design a lightweight hash commitment audit scheme: *i) Commitment phase*: The cryptographic hash of negotiated continuous parameters (allocated CPU frequency, transmit power) is recorded on-chain with the discrete offloading decision. Plaintext parameters are transmitted off-chain; *ii) Verification phase*: After task completion, the requester infers the actual resource allocation from observed end-to-end latency, and compares its hash with the on-chain commitment; *iii) Penalty mechanism*: Verified fraud triggers severe reputation deduction and on-chain credit forfeiture. This scheme ensures full auditability.

2) Critic Network Architecture: During training, a centralized critic network Q_{ϕ_n} for each agent evaluates the joint action of all agents, taking all observations and actions $(\mathbf{o}, \mathbf{a})$ as input to produce a global Q-value. This guides the actors toward cooperative strategies.

3) Training Process: The training follows the standard MADDPG update rules. The critic network Q_{ϕ_n} is updated by minimizing the Mean Squared Bellman Error (MSBE) loss function, as shown below:

$$\mathcal{L}(\phi_n) = \mathbb{E}_{(\mathbf{o},\mathbf{a},\mathbf{r},\mathbf{o}')\sim\mathcal{D}}\left[(y_n - Q_{\phi_n}(\mathbf{o}, \mathbf{a}))^2\right] \tag{24}$$

where the target value y_n is calculated using the target networks, as shown below:

$$y_n = r_n + \gamma Q'_{\phi'_n}(\mathbf{o}', \mathbf{a}')|_{a'_j=\mu'_{\theta'_j}(o'_j)} \tag{25}$$

The actor network μ_{θ_n} is updated using the sampled policy gradient, as shown below:

$$\nabla_{\theta_n} J(\mu) \approx \mathbb{E}_{\mathbf{o}\sim\mathcal{D}}\big[\nabla_{a_n} Q_{\phi_n}(\mathbf{o}, a_1, \ldots, a_N)|_{a_n=\mu_{\theta_n}(o_n)} \nabla_{\theta_n}\mu_{\theta_n}(o_n)\big]. \tag{26}$$

The overall process is detailed in Alg. 1.

D. Computational Complexity Analysis

The computational complexity of the SkyChain Intelligence solution is analyzed by separating it into two distinct phases:

1) Decentralized Execution Complexity: This is the most critical aspect for real-time LAEAI operations. During execution, each agent n only needs to perform a single forward pass through its local actor network μ_{θ_n} to compute its action a_n from its local observation o_n. Let L_a represent the number of parameters in the actor network's layers. The computational complexity for this operation is $\mathcal{O}(L_a)$. Crucially, this complexity is independent of the total number of agents N in the network. This low and constant-time complexity ensures that the decision-making process is highly scalable and suitable for deployment on resource-constrained UAVs.

2) Centralized Training Complexity: The training phase, typically performed offline on a powerful server, bears the majority of the computational load. The complexity is driven by the centralized critic updates. Each of the N critic networks, Q_{ϕ_n}, must process the joint observations $\mathbf{o} = (o_1, \ldots, o_N)$

Algorithm 1: SkyChain Intelligence Training Algorithm

Initialization:

for *each agent* $n = 1, \ldots, N$ **do**

1. Initialize actor μ_{θ_n} and critic Q_{ϕ_n} networks with random weights θ_n, ϕ_n
2. Initialize target networks $\mu'_{\theta'_n}$ and $Q'_{\phi'_n}$ with weights $\theta'_n \leftarrow \theta_n, \phi'_n \leftarrow \phi_n$

3. Initialize shared replay buffer $\mathcal{D}$

for *episode = 1 to MaxEpisodes* **do**

Episode Initialization:

1. Receive initial global state s and local observations $\{o_n\}_{n=1}^N$

for $t = 1$ *to MaxSteps* **do**

Action Selection:

for *each agent* $n = 1, \ldots, N$ **do**

1. Select hybrid action $a_n = (a_n^d, a_n^c)$ from actor $\mu_{\theta_n}(o_n)$ with exploration noise

Environment Interaction:

1. Execute joint action $\mathbf{a} = (a_1, \ldots, a_N)$
2. Observe joint reward $\mathbf{r} = (r_1, \ldots, r_N)$ and next observations $\{o'_n\}_{n=1}^N$
3. Store $(\mathbf{o}, \mathbf{a}, \mathbf{r}, \mathbf{o}')$ in replay buffer $\mathcal{D}$
4. $\mathbf{o} \leftarrow \mathbf{o}'$

Network Update:

for *each agent* $n = 1, \ldots, N$ **do**

1. Sample a random minibatch of S transitions $(\mathbf{o}^i, \mathbf{a}^i, \mathbf{r}^i, \mathbf{o}'^i)$ from $\mathcal{D}$
2. Set target Q-value: $y_n^i = r_n^i + \gamma Q'_{\phi'_n}(\mathbf{o}'^i, \mathbf{a}'^i)|_{a'_j = \mu'_{\theta'_j}(o'_j)}$
3. Update critic by minimizing the loss: $\mathcal{L}(\phi_n) = \frac{1}{S}\sum_i (y_n^i - Q_{\phi_n}(\mathbf{o}^i, \mathbf{a}^i))^2$
4. Update actor using the sampled policy gradient:

$$\nabla_{\theta_n} J \approx \frac{1}{S} \sum_i \nabla_{a_n} Q_{\phi_n}(\mathbf{o}^i, a_1^i, \ldots, a_N^i) \Big|_{a_n = \mu_{\theta_n}(o_n^i)} \cdot \nabla_{\theta_n} \mu_{\theta_n}(o_n^i)$$

Target Network Update:

1. Soft update target networks for each agent n:
2. $\phi'_n \leftarrow \tau\phi_n + (1-\tau)\phi'_n$
3. $\theta'_n \leftarrow \tau\theta_n + (1-\tau)\theta'_n$

and joint actions $\mathbf{a} = (a_1, \ldots, a_N)$ of all N agents. Consequently, the input size of each critic scales linearly with N. Let S be the minibatch size, and L_c be the complexity of a critic network. To update all N critics and N actors for one gradient step, the total computational complexity is approximately $\mathcal{O}(S \cdot N^2 \cdot (L_a + L_c))$. This quadratic scaling $\mathcal{O}(N^2)$ with the number of agents is a known characteristic of MADDPG, representing the trade-off for achieving globally-aware, cooperative policies.

In summary, the SkyChain Intelligence framework intelligently shifts the high computational burden, quadratic scaling $\mathcal{O}(N^2)$ to the offline training phase, while maintaining a low, constant-time complexity, $\mathcal{O}(1)$ with respect to N, for real-time, decentralized execution.

V. Security Analysis

The SkyChain Intelligence framework is designed to be resilient against various security threats prevalent in decentralized multi-agent systems. This section analyzes its robustness against several common attack vectors.

A. Data Tampering and Integrity Attacks

This attack represents a malicious service-providing agent that can execute a task incorrectly, either intentionally or due to a fault, and return a false result to the requester to save computational resources [49].

Our framework counters this threat using the blockchain's inherent properties of immutability and transparency. When a task is completed, the service provider submits the hash of the computation result, which is recorded in an immutable transaction on the blockchain. The task requester can verify the result and submit feedback, success or failure, to the reputation smart contract. Any attempt by the provider to tamper with the result would produce a different hash, which the requester can easily detect. Consistent submission of incorrect results will lead to negative feedback, causing the malicious agent's reputation score to plummet. The DRL agents, driven by the reputation-fused reward function, will learn to avoid offloading tasks to this untrustworthy node, effectively isolating it from the network's economy.

B. Sybil Attacks

It is an adversary that can create a large number of pseudonymous identities, namely Sybil nodes, to gain a disproportionate influence on the network, manipulate the reputation system, or launch coordinated attacks.

The SkyChain framework employs a two-layered defense. First, the use of a consortium blockchain acts as a primary barrier. Unlike public blockchains, nodes cannot join anonymously. They must be authenticated and granted permission to participate. This makes creating a large number of fake identities significantly more difficult and costly for an attacker. Second, even if an attacker manages to introduce a few malicious nodes, the reputation system provides further resilience. New nodes start with a neutral or low reputation and must build trust over time through reliable interactions. The cost and time required to build a positive reputation for a multitude of Sybil nodes make it economically infeasible to manipulate the system on a large scale.

C. Selfish Behavior and Free-Riding

Rational but selfish agents might agree to provide a service but then fail to execute the task or execute it with minimal resources, to conserve their energy and computility, a behavior known as free-riding [50].

The synergistic link between the blockchain's reputation score and the MADRL's reward function directly disincentivizes such behavior. An agent that consistently fails to

complete tasks will receive negative feedback, leading to a low reputation score. Since other agentic AIs are optimizing for a reward that includes the reputation of their partners, they will quickly learn that selecting a selfish, low-reputation node results in a lower cumulative reward. Consequently, selfish agents will receive fewer and fewer offloading requests, denying them the potential benefits of participating in the computational market and isolating them from the cooperative network.

D. Denial of Service (DoS) Attacks

An attacker could attempt to disrupt the network by flooding a high-reputation, critical service-providing aircraft with a large volume of malicious or fake task requests, aiming to overwhelm its resources and render it unavailable to legitimate users [51].

While the blockchain layer does not directly prevent DoS attacks, the agentic AI layer provides significant resilience through adaptive decision-making. Each agent's observation space includes local metrics like task queue length and experienced latency. When a node is under a DoS attack, its processing and queuing delays will increase dramatically. Neighboring agents will observe this degradation in performance. Their MADRL policies, trained to minimize overall task completion latency, will naturally learn to avoid the congested node and reroute their offloading requests to other available, responsive agents. This intelligent, decentralized load-balancing behavior effectively mitigates the impact of the DoS attack on the overall network performance without requiring a central authority to detect and block the attack.

VI. Performance Evaluation

This section provides a comprehensive evaluation of the proposed SkyChain Intelligence framework through extensive simulation experiments using a MATLAB-NS-3 co-simulation architecture, where NS-3 emulates realistic wireless propagation, UAV mobility and blockchain consensus interactions, and MATLAB implements the MADRL algorithm and system performance calculation. It runs on a server equipped with three 96-core Intel(R) Xeon(R) Gold 5220R CPUs, 1 TB of memory, and 8 NVIDIA GeForce RTX 3090 GPUs.

A. Simulation Setup and Comparison Schemes

We simulate a 3D environment over a 2000 m x 2000 m 2D area, containing 50 randomly distributed ground task generators. The simulation proceeds in discrete time slots, running for a total of 2000 slots. All simulation results are averaged over 50 independent experiments to ensure statistical validity. The other key parameters used in the simulation are detailed in Table III.

To validate the superiority of our proposed SkyChain framework, we compare it with the following three comparison schemes: 1) Game-Theoretic Offloading (GTO) [21] is a non-cooperative game-theoretic approach in which each LAEAI agent acts selfishly to maximize its utility; 2) FL-Aided Offloading (FLAO) [20] is a scheme where agents use FL to collaboratively train a shared neural network model that predicts the expected latency and energy cost for offloading a task to any other agent in the network; 3) DRL without Blockchain (DRL-noBC) [52] uses a similar MADDPG algorithm without the blockchain layer and trust mechanism, but retains the core of Agentic AI. 4) Blockchain-Empowered MADRL (BC-MADRL) [53] is a homologous blockchain-aided multi-agent reinforcement learning baseline adapted for computation offloading. It leverages a trust-enhanced PBFT consensus protocol to maintain dynamic node reputation scores.

TABLE III: Simulation Parameters

Parameter	Value
Network Settings	
Area Size	2000×2000 m^2
Number of Aircraft	5, 10, 15, 20, 25
Percentage of Malicious aircraft	10%
Number of Task Generators	50
Aircraft Flight Altitude (H)	Uniformly in [50, 100] m
Simulation Time Slots	2000
Communication Parameters	
Channel Model	Probabilistic LoS (Urban)
Path Loss Parameters (a, b)	9.61, 0.16
Bandwidth (B)	20 MHz
Transmit Power (P_m)	0.1 W
Noise Power Spectral Density (N_0)	-174 dBm/Hz
Shadowing Std. (LoS / NLoS)	4 dB / 12 dB
Rician K-factor (A2A LoS)	20 dB
Computation Task Parameters	
Task Data Size (D_k)	Uniformly in [0.5, 2.0] Mbits
Task Computation Density (C_k)	Uniformly in [500, 2500] cycles/bit
Task Arrival Rate (λ)	Uniformly in [0.1, 0.9] tasks/slot/device
Aircraft CPU Frequency (f_n)	Uniformly in [1.0, 2.0] GHz
Ariel Model Parameters	
Max Speed ($v_{\max}$)	25 m/s
Propulsion Power Params (P_0, P_i)	79.8 W, 88.6 W
Min Safety Distance ($d_{\min}$)	50 m
Completion Reliability (ε_k)	0.95
Blockchain Parameters	
Block Interval	2 s
Transaction Size	256 Bytes
Consensus Message Size (S_{msg})	128 Bytes
Minimum Consensus Rate ($R_{\min}$)	1 Mbps
Local Validation Energy ($E_{\text{comp}}^{\text{BC}}$)	0.002 J
Shard LoS Threshold ($P_{\text{th}}^{\text{shard}}$)	0.8
Short-term Reputation Factor (β_s)	0.3
Long-term Reputation Factor (β_l)	0.05
Reputation Fusion Weight (λ)	0.4
Checkpoint interval (K)	10 blocks
MADDPG Parameters	
Actor/Critic Network	3-layer MLP (256-256-128)
Learning Rate (Actor/Critic)	1×10^{-4} / 1×10^{-3}
Discount Factor (γ)	0.99
Replay Buffer Size	1×10^6
Batch Size	1024
Objective Weights (w_D, w_E, w_R)	(0.4, 0.4, 0.2)

B. Convergence comparison

We verify the convergence of SkyChain Intelligence by comparing it with DRL-noBC and MADDPG from [18] in three scenarios. The three scenarios are as follows. 1) Baseline normal scenario: 20 aircraft, 10% malicious nodes, task arrival rate of $\lambda = 0.5$ tasks/slot/device; 2) High network load scenario: 20 aircraft, 10% malicious nodes, task arrival rate $\lambda = 0.9$ tasks/slot/device; 3) High security threat scenario:

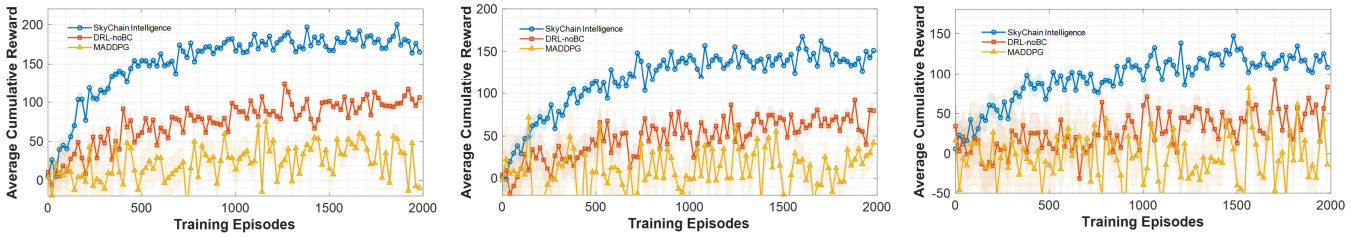


Fig. 4: Convergence comparison. (a) Baseline normal scenario; (b) High network load scenario; (c). High security threat scenario.

20 aircraft, 20% malicious nodes, task delivery rate $\lambda = 0.5$ tasks/slot/device. The result is shown in Fig. 4.

As shown in Fig. 4 (a), in the baseline conventional scenario, SkyChain Intelligence has the fastest convergence speed, reaching 95% stable rewards after 300 rounds. Its training process shows no significant fluctuations. The convergence speed of DRL-noBC is 60% slower than that of SkyChain Intelligence, and the stable reward is only 55.6% of that of SkyChain Intelligence. MADDPG has even worse convergence performance, with a mean stable reward of only 40 and a highly fluctuating training process. As shown in Fig. 4 (b), in the high network load scenario, SkyChain Intelligence still maintains stable convergence. After 350 rounds, it reaches 95% steady-state rewards, with a small fluctuation range that is controllable. The convergence speed and performance of DRL-noBC deteriorate significantly. After 650 rounds, it reaches the convergence threshold, which is 30% lower than the benchmark scenario. MADDPG completely stagnates in convergence, and no effective strategy learning occurred within 2000 rounds. As shown in Fig. 4 (c), in the high security threat scenario, the convergence performance of SkyChain Intelligence slightly deteriorated but remained stable. It reaches 95% steady-state reward after 400 rounds. Compared to the baseline scenario, it only decreases by 33.3%, achieving precise isolation of malicious nodes. The performance of DRL-noBC plummets. It takes 880 rounds to reach the convergence threshold. MADDPG failed to converge, deteriorating by 87.5% compares to the baseline scenario and failing strategy learning.

The reason SkyChain Intelligence has an advantage in convergence is as follows: First, dynamic reputation scoring on the blockchain enables the active isolation of malicious nodes. This eliminates the high noise and strong randomness of the reward signals caused by task failures and retransmissions; Secondly, embodied AI allows agents to autonomously adapt to the high dynamics and non-stationary characteristics of low-altitude networks, avoiding the strategy failure and convergence oscillation caused by environmental dynamic changes; Thirdly, the dual-head actor-critic architecture breaks through the action space limitations of the MADDPG. It can achieve end-to-end global optimization of discrete offloading decisions and continuous resource allocation. Also, it directly embeds the on-chain reputation score into the reward function, which can simultaneously improve the convergence speed.

C. Performance Comparison

We present and analyze the simulation results under the 10% of the aircraft are malicious. These malicious agents accept offloading tasks but have a high probability (90%) of dropping them to save their resources, effectively failing the task. This scenario is designed to test the robustness and practical value of our framework in the open wireless LACNets.

1) Average Task Completion Delay: As shown in Fig. 5 (a), as the number of aircraft increases, the average latency for SkyChain decreases significantly, as it learns to route tasks to the growing pool of reliable nodes. BC-MADRL, as a blockchain-aided baseline, also outperforms all trust-agnostic schemes, but its latency remains consistently higher because it only incorporates trust into the observation space without reward-level deep fusion. In stark contrast, the latency for DRL-noBC, GTO, and FLAO remains high and decreases much more slowly. These trust-agnostic schemes frequently offload tasks to malicious nodes, leading to task failures and necessitating re-transmissions, which severely inflates their average latency. SkyChain's ability to identify and avoid malicious nodes gives it a decisive performance advantage. As shown in Fig. 5 (b), with increasing network load, the latency of all schemes rises. However, the performance gap between SkyChain and the other schemes widens dramatically. BC-MADRL sees moderate latency growth, but under high load, its insufficient trust incentive leads to more misselections of low-reputation nodes, enlarging the gap with our proposal. For DRL-noBC, GTO, and FLAO, a higher load means that more tasks are inevitably sent to malicious nodes, resulting in a catastrophic rise in effective latency. SkyChain, however, consistently routes tasks to trusted nodes, and its latency increases gracefully due to normal network congestion, showcasing its superior robustness under pressure. As shown in Fig. 5 (c), as tasks become more computationally intensive, the cost of a failed offload becomes much higher. Consequently, the latency of DRL-noBC, GTO, and FLAO skyrockets. BC-MADRL mitigates most malicious failures via its basic reputation mechanism, but lacks hybrid-action joint optimization for precise resource matching, so its latency grows faster than SkyChain. SkyChain, by ensuring tasks are sent to reliable nodes from the outset, avoids the crippling penalty of failed complex tasks, demonstrating a clear and widening performance gap as task complexity increases.

2) Average System Energy Consumption: In Fig. 6 (a), it demonstrates that the total system energy for SkyChain has

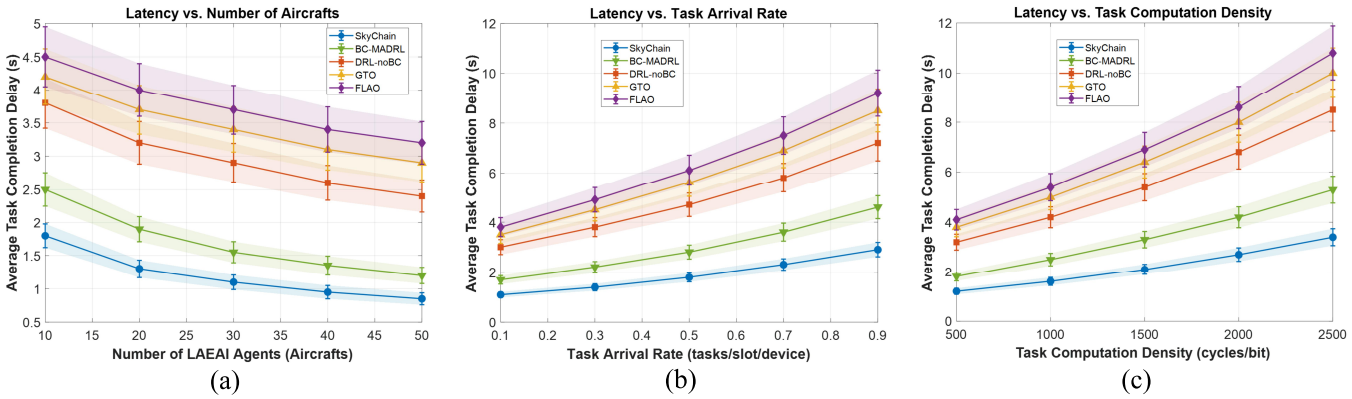


Fig. 5: Average task completion delay. (a) Under different numbers of LAEAI agents; (b) Under different task arrival rates; (c) Under different task computation densities.

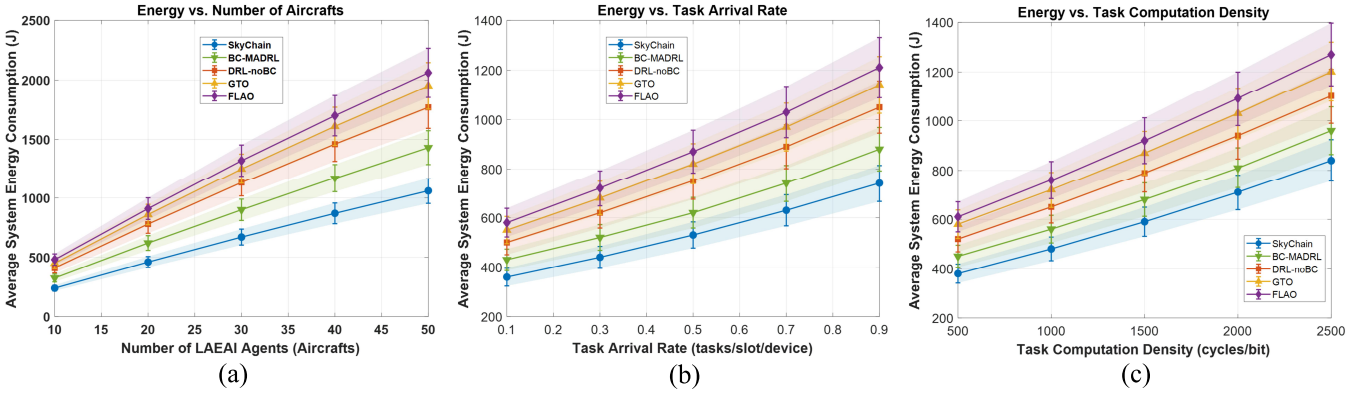
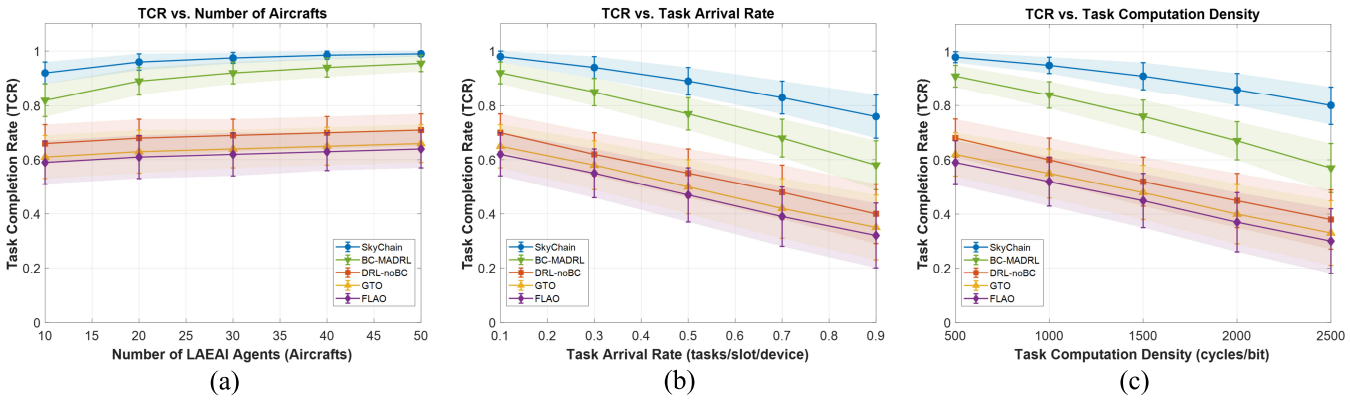


Fig. 6: Average system energy consumption. (a) Under different numbers of LAEAI agents; (b) Under different task arrival rates; (c) Under different task computation densities.

a moderate, linear increase with more aircraft. BC-MADRL outperforms trust-agnostic schemes but consumes more energy than SkyChain, as its basic trust mechanism only partially eliminates retransmission waste. The energy consumption for DRL-noBC, GTO, and FLAO is significantly higher. This is because every task offloaded to a malicious node wastes the transmission energy and requires re-transmission, doubling the energy cost for that task. SkyChain avoids this wasted energy by making reliable offloading decisions, thus achieving much higher energy efficiency for the entire system. As the task rate increases, the energy wastage of the trust-agnostic schemes is amplified, which is illustrated in Fig. 6 (b). BC-MADRL sees a moderate growth rate, but its shallow trust integration leads to accumulating retransmission overhead under high load. More tasks being sent to malicious nodes leads to a rapid increase in total energy consumption for DRL-noBC, GTO, and FLAO. SkyChain's energy consumption increases at a much slower rate, as it primarily expends energy on productive computations, not on failed and repeated transmissions. The energy difference is less pronounced with task density, as computation energy at the malicious node is not expended by the requester, as shown in Fig. 6 (c). BC-MADRL narrows the gap with SkyChain, but residual retransmission waste still keeps its energy consumption higher. However, the energy wasted on transmission and re-transmission still keeps SkyChain as the most energy-efficient solution, especially when considering the energy cost of not completing high-value tasks.

3) Task Completion Rate: As shown in Fig. 7 (a), this metric most clearly demonstrates the value of SkyChain.this metric most clearly demonstrates the value of SkyChain. The TCR for SkyChain remains consistently high (above 90%) and improves with more UAVs. BC-MADRL achieves a moderate completion rate, as its basic reputation mechanism filters most malicious nodes, but still lags behind SkyChain due to the lack of trust-performance joint optimization. Conversely, the TCR for DRL-noBC, GTO, and FLAO hovers at a much lower level. Since 10% of the nodes are malicious and these schemes choose offloading targets without regard to trust, they are statistically prone to a high rate of failure, which our simulation confirms. As network load increases, the TCR for the trust-agnostic schemes plummets, as demonstrated in Fig. 7 (b). BC-MADRL sees moderate degradation, but its shallow trust integration cannot fully avoid mis-selection of low-reputation nodes under heavy load. Malicious nodes continue to drop tasks, and network congestion causes even legitimate tasks to time out. SkyChain, by efficiently using only the reliable portion of the network, maintains a much more stable and higher TCR, proving its reliability in a hostile environment. Similarly, as shown in Fig. 7 (a), as task complexity rises, the trust-agnostic schemes fail more often due to both malicious activity and their inability to manage resources for demanding tasks. BC-MADRL mitigates partial malicious failures but lacks precise resource matching via hybrid action spaces, leading to more timeouts for computation-intensive



(a) (b) (c)

Fig. 7: Task completion rate. (a) Under different numbers of LAEAI agents; (b) Under different task arrival rates; (c) Under different task computation densities.



(a) (b) (c)

Fig. 8: System trust score. (a) Under different numbers of LAEAI agents; (b) Under different task arrival rates; (c) Under different task computation densities.

tasks. SkyChain's ability to ensure that complex tasks are sent to reliable and capable nodes allows it to maintain a significantly higher completion rate, highlighting its suitability for mission-critical applications.

4) System Trust Score: As shown in Fig. 8 (a), the System Trust Score for SkyChain is consistently high (around 0.95), demonstrating that its agents have successfully learned to select high-reputation partners. BC-MADRL maintains a moderately high trust level (around 0.88), as its basic reputation mechanism also guides agents to prefer trustworthy nodes, but the absence of reward-level trust incentive prevents it from achieving optimal trust selection. For DRL-noBC, GTO, and FLAO, the trust score is consistently low (around $0.45 - 0.5$). This is because they have no mechanism to differentiate between trustworthy and malicious nodes, and their offloading decisions are essentially random with respect to trust. In an environment with 10% malicious nodes, their choices often fall on untrustworthy agents. As shown in Fig. 8 (b), the trust score for all schemes remains relatively stable regardless of the task arrival rate. SkyChain maintains its high trust score, showing that its agents do not compromise on security even when the network is busy. BC-MADRL also retains its moderate trust level without obvious degradation under heavy load. The other schemes remain at their low trust score, continuously making poor, trust-agnostic decisions. As with the other factors, the trust score remains constant. As shown in Fig. 8 (c), the results are stark: SkyChain builds and maintains a highly trustworthy interaction environment, BC-MADRL delivers limited trust improvement via basic blockchain mechanisms, while the other schemes operate in a low-trust environment, making them vulnerable and unreliable. This clearly validates the effectiveness and necessity of deeply fusing the blockchain-based reputation system into the DRL learning framework, rather than simply combining the two technologies.

D. Ablation Experiment

To verify the necessity of the multiple components for SkyChain, we conduct ablation experiments. They are respectively represented as w/o blockchain, w/o trust reward, w/o Agentic AI, and w/o hybrid action, signifying the absence of blockchain, trust rewards, the perception-reasoning-planning-execution-adaptive closed loop of embodied AI, and the hybrid action space. The ablation test scenarios are consistent with the scenarios used in the convergence comparison.

This set of ablation experiments validates the irreplaceability of each core component of SkyChain Intelligence. In all scenarios, the full SkyChain model achieves optimal performance across all four metrics. As shown in Table IV, in the baseline normal scenario, it delivers an average delay of 1.6 ± 0.2 s, average energy consumption of 49025 J, TCR of $94.1\% \pm 1.2\%$, and system trust score of 0.95 ± 0.02. Removing any single component breaks the "Trust-Performance-Overhead" trilemma balance, with performance degradation following a clear severity pattern: removing the blockchain

TABLE IV: Results of the Ablation Experiment in Baseline Normal Scenario

Models	Aver. delay(s)	Aver. energy consumption (J)	Task completion rate	System trust score
SkyChain	1.6 ± 0.2	490 ± 25	$94.1\% \pm 1.2\%$	0.95 ± 0.02
w/o Trust Reward	2.3 ± 0.3	650 ± 35	$83.5\% \pm 1.8\%$	0.74 ± 0.04
w/o Hybrid Action	2.9 ± 0.3	720 ± 30	$78.4\% \pm 2.0\%$	0.70 ± 0.05
w/o Agentic AI	3.4 ± 0.4	810 ± 40	$72.6\% \pm 2.2\%$	0.65 ± 0.05
w/o Blockchain	5.2 ± 0.5	1080 ± 45	$54.2\% \pm 2.8\%$	0.47 ± 0.03

TABLE V: Results of the Ablation Experiment in High Network Load Scenario

Models	Aver. delay(s)	Aver. energy consumption (J)	Task completion rate	System trust score
SkyChain	2.0 ± 0.2	550 ± 30	$90.5\% \pm 1.6\%$	0.93 ± 0.03
w/o Trust Reward	3.0 ± 0.3	740 ± 40	$78.2\% \pm 2.2\%$	0.70 ± 0.05
w/o Hybrid Action	3.7 ± 0.4	820 ± 35	$72.1\% \pm 2.4\%$	0.66 ± 0.06
w/o Agentic AI	4.5 ± 0.5	950 ± 45	$63.8\% \pm 2.7\%$	0.60 ± 0.06
w/o Blockchain	6.3 ± 0.6	1210 ± 55	$45.7\% \pm 3.3\%$	0.44 ± 0.04

TABLE VI: Results of the Ablation Experiment in High Security Threat Scenario

Models	Aver. delay(s)	Aver. energy consumption (J)	Task completion rate	System trust score
SkyChain	2.2 ± 0.3	580 ± 35	$88.7\% \pm 1.8\%$	0.92 ± 0.03
w/o Trust Reward	3.6 ± 0.4	830 ± 45	$71.4\% \pm 2.5\%$	0.65 ± 0.05
w/o Hybrid Action	4.1 ± 0.4	890 ± 40	$67.5\% \pm 2.6\%$	0.63 ± 0.06
w/o Agentic AI	4.9 ± 0.5	1020 ± 50	$59.2\% \pm 2.9\%$	0.57 ± 0.06
w/o Blockchain	7.1 ± 0.7	1380 ± 60	$38.2\% \pm 3.5\%$	0.41 ± 0.04

layer causes the most catastrophic collapse, e.g., TCR drops to $38.2\% \pm 3.5\%$ and delay surges to 7.1 ± 0.7 s in the high security threat scenario, followed by removing trust-reward fusion, hybrid action space optimization, and Agentic AI. Notably, environmental complexity amplifies degradation. As shown in Table V and VI, in the high network load and high security threat scenarios, all variants show significantly worse performance, while the full SkyChain model remains robust, with only minor fluctuations. For instance, TCR stays above 88% and trust score above 0.92 in the high threat scenario.

VII. Conclusion

This paper has investigated the core challenges of enabling autonomous, secure, and efficient collaborative LAEAI agents in decentralized LACNets. We have identified and formalized a fundamental "Trust-Performance-Overhead" trilemma that plagues existing solutions, and proposed SkyChain Intelligence, an innovative framework to address this issue. Through comprehensive simulation evaluations, convergence comparison, and ablation experiments, we have validated the superiority and robustness of the proposed SkyChain Intelligence. It has consistently achieved lower end-to-end latency, higher energy efficiency, and greater TSR reliability than advanced baseline schemes. Meanwhile, our framework has maintained fast and stable convergence across complex environments, and ablation studies have verified the indispensable role of each core component. This work has demonstrated that deeply integrating verifiable trust mechanisms into the learning core of agentic AI can enable decentralized autonomous systems that are both efficient and trustworthy.

References

[1] Z. Li, Z. Gao, K. Wang, Y. Mei, C. Zhu, L. Chen, X. Wu, and D. Niyato, "Unauthorized uav countermeasure for low-altitude economy: Joint communications and jamming based on mimo cellular systems," *IEEE Internet of Things Journal*, vol. 12, no. 6, pp. 6659–6672, 2025.

[2] L. Cai, R. Zhang, C. Zhao, Y. Zhang, J. Kang, D. Niyato, T. Jiang, and X. Shen, "Large language model-enhanced reinforcement learning for low-altitude economy networking," *arXiv preprint arXiv:2505.21045*, 2025.

[3] Y. Yang, Y. Chen, J. Wang, G. Sun, and D. Niyato, "Embodied ai-empowered low altitude economy: Integrated sensing, communications, computation, and control (isc3)," *arXiv preprint arXiv:2412.19996*, 2024.

[4] R. Zhang, H. Du, Y. Liu, D. Niyato, J. Kang, Z. Xiong, A. Jamalipour, and D. I. Kim, "Generative ai agents with large language model for satellite networks via a mixture of experts transmission," *IEEE Journal on Selected Areas in Communications*, vol. 42, no. 12, pp. 3581–3596, 2024.

[5] Y. Xue, R. Liang, B. Yang, X. Cao, Z. Yu, M. Debbah, and C. Yuen, "Joint task offloading and resource allocation in low-altitude mec via graph attention diffusion," *IEEE Transactions on Cognitive Communications and Networking*, 2026.

[6] H. Luo, Y. Liu, R. Zhang, J. Wang, G. Sun, D. Niyato, H. Yu, Z. Xiong, X. Wang, and X. Shen, "Toward edge general intelligence with multiple-large language model (multi-llm): Architecture, trust, and orchestration," *IEEE Transactions on Cognitive Communications and Networking*, 2025.

[7] H. Luo, R. Zhang, Y. Liu, G. Sun, H. Yu, and Z. Han, "Real world assets on-chain assistance low-altitude computility networks: Architecture, methodology, and challenges," *IEEE Internet of Things Magazine*, 2026.

[8] Z. Wang, T. Wei, G. Sun, X. Liu, H. Yu, and D. Niyato, "Multi-uav enabled mec networks: Optimizing delay through intelligent 3-d trajectory planning and resource allocation," *IEEE Transactions on Intelligent Transportation Systems*, 2025.

[9] Q. Chen, W. Meng, T. Q. Quek, and S. Chen, "Multi-tier hybrid offloading for computation-aware iot applications in civil aircraft-augmented sagin," *IEEE Journal on Selected Areas in Communications*, vol. 41, no. 2, pp. 399–417, 2022.

[10] Z. Wang, G. Sun, Y. Wang, D. Niyato *et al.*, "Cluster-based multi-agent task scheduling for space-air-ground integrated networks," *IEEE Transactions on Cognitive Communications and Networking*, 2025.

[11] Y. Liu, G. Liu, J. Wang, R. Zhang, D. Niyato, G. Sun, Z. Xiong, and Z. Han, "Lameta: Intent-aware agentic network optimization via a large ai model-empowered two-stage approach," *IEEE Journal on Selected Areas in Communications*, 2026.

[12] Y. Sun, Y. Liu, S. Guo, X. Qiu, J. Chen, J. Hao, and D. Niyato,

"Edge large ai model agent-empowered cognitive multimodal semantic communication," *IEEE Transactions on Mobile Computing*, 2025.

[13] H. Luo, G. Sun, C. Chi, H. Yu, and M. Guizani, "Convergence of symbiotic communications and blockchain for sustainable and trustworthy 6g wireless networks," *IEEE Wireless Communications*, vol. 32, no. 2, pp. 18–25, 2025.

[14] H. Luo, G. Sun, Y. Liu, D. Zhao, D. Niyato, H. Yu, and S. Dustdar, "A weighted byzantine fault tolerance consensus driven trusted multiple large language models network," *IEEE Transactions on Cognitive Communications and Networking*, 2025.

[15] R. Quirynen, S. Safaoui, and S. Di Cairano, "Real-time mixed-integer quadratic programming for vehicle decision-making and motion planning," *IEEE Transactions on Control Systems Technology*, vol. 33, no. 1, pp. 77–91, 2025.

[16] Z. Kang, C. You, and R. Zhang, "3d placement for multi-uav relaying: An iterative gibbs-sampling and block coordinate descent optimization approach," *IEEE Transactions on Communications*, vol. 69, no. 3, pp. 2047–2062, 2020.

[17] M. Fu, Y. Shi, and Y. Zhou, "Federated learning via unmanned aerial vehicle," *IEEE Transactions on Wireless Communications*, vol. 23, no. 4, pp. 2884–2900, 2023.

[18] G. Chen, F. Sun, H. Liang, Q. Zeng, and Y.-D. Zhang, "Maddpg-m&l: Uav-assisted joint user association and slicing resource allocation in hetnets," *IEEE Transactions on Network Science and Engineering*, 2025.

[19] Z. Wang, J. Chen, G. Sun, H. Yu, and M. Guizani, "Joint energy-efficient task scheduling and trajectory optimization for multi-uav mec networks in low-altitude economy," *IEEE Transactions on Cognitive Communications and Networking*, 2025.

[20] V. M. R. Tummala, A. Hazra, A. Kalita, and M. Gurusamy, "Efficient task offloading through federated learning in uav-assisted edge networks," in *2024 IEEE 99th Vehicular Technology Conference (VTC2024-Spring)*. IEEE, 2024, pp. 1–5.

[21] Y. Chen, J. Zhao, Y. Wu, J. Huang, and X. Shen, "Multi-user task offloading in uav-assisted leo satellite edge computing: A game-theoretic approach," *IEEE Transactions on Mobile Computing*, vol. 24, no. 1, pp. 363–378, 2024.

[22] Z. Jia, S. He, L. Yuan, F. Zhou, Q. Wu, Z. Han, and D. Niyato, "Blockchain-enabled routing for zero-trust low-altitude intelligent networks," *IEEE Journal on Selected Areas in Communications*, 2026.

[23] M. E. Mkiramweni, C. Yang, J. Li, and W. Zhang, "A survey of game theory in unmanned aerial vehicles communications," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 4, pp. 3386–3416, 2019.

[24] R. Zheng, Y. Zheng, Z. Cheng, L. Luo, H. Luo, G. Sun, H. Yu, and D. Niyato, "Agentvne: Llm-augmented graph reinforcement learning for affinity-aware multi-agent placement in edge agentic ai," *arXiv preprint arXiv:2601.02021*, 2026.

[25] L. Cai, J. Wang, R. Zhang, Y. Zhang, T. Jiang, D. Niyato, X. Wang, A. Jamalipour, and X. Shen, "Secure physical layer communications for low-altitude economy networking: A survey," *IEEE Communications Surveys & Tutorials*, 2025.

[26] F. Frattolillo, D. Brunori, and L. Iocchi, "Scalable and cooperative deep reinforcement learning approaches for multi-uav systems: A systematic review," *Drones*, vol. 7, no. 4, p. 236, 2023.

[27] Z. Zheng and S. Gu, "Safe multi-agent reinforcement learning with bilevel optimization in autonomous driving," *IEEE Transactions on Artificial Intelligence*, vol. 6, no. 4, pp. 829–842, 2025.

[28] Y. Lin, Z. Gao, H. Du, J. Kang, D. Niyato, Q. Wang, J. Ruan, and S. Wan, "Drl-based adaptive sharding for blockchain-based federated learning," *IEEE Transactions on Communications*, vol. 71, no. 10, pp. 5992–6004, 2023.

[29] H. Luo, Q. Zhang, G. Sun, H. Yu, and D. Niyato, "Symbiotic blockchain consensus: Cognitive backscatter communications-enabled wireless blockchain consensus," *IEEE/ACM Transactions on Networking*, vol. 32, no. 6, pp. 5372–5387, 2024.

[30] H. Luo, G. Sun, J. Wang, H. Yu, D. Niyato, S. Dustdar, and Z. Han, "Wireless blockchain meets 6g: The future trustworthy and ubiquitous connectivity," *IEEE Communications Surveys and Tutorials*, 2025.

[31] Y. Liu, H. Du, D. Niyato, J. Kang, Z. Xiong, C. Miao, X. Shen, and A. Jamalipour, "Blockchain-empowered lifecycle management for ai-generated content products in edge networks," *IEEE Wireless Communications*, vol. 31, no. 3, pp. 286–294, 2024.

[32] L. Che, J. Wang, X. Liu, and F. Ma, "Leveraging foundation models for multi-modal federated learning with incomplete modality," in *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, 2024, pp. 401–417.

[33] L. Che, J. Wang, Y. Zhou, and F. Ma, "Multimodal federated learning: A survey," *Sensors*, vol. 23, no. 15, p. 6986, 2023.

[34] X. Yuan, H. Tian, X. Zhang, H. Du, N. Zhang, K. Huang, and L. Cai, "Digital twin-driven madrl approaches for communication-computing-control co-optimization," *IEEE Journal on Selected Areas in Communications*, 2025.

[35] M. Gapeyenko, D. Moltchanov, S. Andreev, and R. W. Heath, "Line-of-sight probability for mmwave-based uav communications in 3d urban grid deployments," *IEEE Transactions on Wireless Communications*, vol. 20, no. 10, pp. 6566–6579, 2021.

[36] A. Al-Hourani *et al.*, "Optimal lap altitude for maximum coverage," *IEEE Wireless Communications Letters*, vol. 3, no. 6, pp. 569–572, 2014.

[37] A. M. Salhab and M. H. Samuh, "Accurate performance analysis of reconfigurable intelligent surfaces over rician fading channels," *IEEE Wireless Communications Letters*, vol. 10, no. 5, pp. 1051–1055, 2021.

[38] B. Li *et al.*, "Measurement-based modeling of air-to-air channels for small UAVs in urban environments," *IEEE Internet of Things Journal*, vol. 9, no. 17, pp. 16 321–16 332, 2022.

[39] Y. Zeng *et al.*, "Wireless communications with unmanned aerial vehicles: Opportunities and challenges," *IEEE Communications magazine*, vol. 54, no. 5, pp. 36–42, 2016.

[40] C. Liu, Y. Zhong, R. Wu, S. Ren, S. Du, and B. Guo, "Deep reinforcement learning based 3d-trajectory design and task offloading in uav-enabled mec system," *IEEE Transactions on Vehicular Technology*, vol. 74, no. 2, pp. 3185–3195, 2025.

[41] A. Goldsmith, *Wireless Communications*. Cambridge University Press, 2005.

[42] H. Liang, L. Zhu, F. R. Yu, and X. Wang, "A cross-layer defense method for blockchain empowered cbtc systems against data tampering attacks," *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 1, pp. 501–515, 2022.

[43] Y. Liu, K. Qian, K. Wang, and L. He, "Bcmaster: A compatible framework for comprehensively analyzing and monitoring blockchain systems in iot," *IEEE Internet of Things Journal*, vol. 9, no. 22, pp. 22 529–22 546, 2022.

[44] S. Guo, Y. Wang, Z. Su, Y. Pan, T. H. Luan, and X. Luo, "Flet: Game-theoretic free-riding mitigation via test tasks in federated learning," *IEEE Transactions on Networking*, 2026.

[45] Y.-L. Zhi, Z. Nie, X. Liu, and S. He, "Hybrid event-triggered $H\ \infty$ control for networked control systems under denial of service attacks," *IEEE Transactions on Network Science and Engineering*, 2025.

[46] X. Gao, Y. Wang, B. Liu, X. Zhou, R. Zhang, J. Wang, D. Niyato, D. I. Kim, A. Jamalipour, C. Yuen *et al.*, "Agentic satellite-augmented low-altitude economy and terrestrial networks: A survey on generative approaches," *arXiv preprint arXiv:2507.14633*, 2025.

[47] R. Zhang, S. Tang, Y. Liu, D. Niyato, Z. Xiong, S. Sun, S. Mao, and Z. Han, "Toward agentic ai: generative information retrieval inspired intelligent communications and networking," *IEEE Communications Magazine*, 2026.

[48] A. Kopic, E. Perenda, and H. Gacanin, "A collaborative multi-agent deep reinforcement learning-based wireless power allocation with centralized training and decentralized execution," *IEEE Transactions on Communications*, vol. 72, no. 11, pp. 7006–7016, 2024.

[49] R. Jia, P. Yu, F.-W. Jing, and J. Guo, "Parameter identification algorithm for fir systems with quantized inputs and binary outputs under data tampering attacks," *IEEE Transactions on Instrumentation and Measurement*, vol. 73, pp. 1–12, 2024.

[50] T. Chen, F. Wang, W. Qiu, Q. Zhang, Z. Xiong, and Z. Zheng, "Toward free-riding attack on cross-silo federated learning through evolutionary game," in *2024 IEEE 44th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2024, pp. 869–880.

[51] R. Uddin, S. A. Kumar, and V. Chamola, "Denial of service attacks in edge computing layers: Taxonomy, vulnerabilities, threats and solutions," *Ad Hoc Networks*, vol. 152, p. 103322, 2024.

[52] X. Tang, Q. Chen, W. Weng, C. Jin, Z. Liu, J. Wang, G. Sun, X. Li, and D. Niyato, "Task assignment and exploration optimization for low altitude uav rescue via generative ai enhanced multi-agent reinforcement learning," *IEEE Transactions on Mobile Computing*, vol. 25, no. 1, pp. 627–643, 2026.

[53] Z. Jia, S. He, Q. Zhu, W. Wang, Q. Wu, and Z. Han, "Trusted routing for blockchain-empowered uav networks via multi-agent deep reinforcement learning," *IEEE Transactions on Communications*, 2025.